



NCRD's Sterling Institute of Management Studies
(NAAC Accredited A+ Grade)

Nerul, Navi Mumbai

2025-2027

GENERATIVE AI 10-Day Workshop

Project Documentation

On

AI Chatbot Integration using OpenRouter API

Under the Guidance

of

Mr. Navneet Sir

By

Name : Aniket Lad

Roll No : 94

SYMCA Div:B

Title

Building an LLM-Powered AI Chatbot using Python and the OpenRouter API

Aim

To understand the fundamentals of Generative AI by developing and configuring a Python-based chatbot that interacts with Large Language Models (LLMs) via an API proxy network.

Objectives

- To explore Generative AI fundamentals and understand how LLMs generate text.
- To comprehend core concepts like tokens, embeddings, and context window management.
- To implement a working Python application capable of sending inference requests to the NVIDIA Nemotron 3 Ultra model through OpenRouter.
- To securely manage API credentials within a Google Colab environment.

Theory & Background Knowledge

Before analyzing the code, it is essential to understand the underlying Generative AI framework that powers the chatbot:

- **Generative AI & LLMs:** Generative AI refers to systems capable of generating new content (text, code, images). Large Language Models (LLMs) are a subset of this, specifically trained on vast datasets of text to understand and generate human-like language based on statistical probability.
- **Transformer Architecture (High-Level):** Modern LLMs rely on the Transformer architecture. Instead of reading text sequentially, Transformers use a mechanism called *self-attention* to weigh the importance of all words in a sentence simultaneously, allowing the model to capture deep semantic context.
- **Tokens and Tokenization:** LLMs do not read words; they read "tokens." Tokenization is the process of breaking down text into smaller sub-word units.
- **Context Window:** The context window is the maximum number of tokens an LLM can process in a single request (both the input prompt and the generated output combined). In our implementation, the selected NVIDIA Nemotron 3 Ultra model is accessed through OpenRouter, allowing the prompt and generated response to be handled through the same API workflow.
- **Parameters, Training, and Inference:** A model's "parameters" are the billions of adjustable weights it learned during its initial *Pre-training* phase. Our Python code executes the *Inference* phase—where the trained model processes our prompt and generates a response in real-time.
- **NVIDIA Nemotron 3 Ultra Model:** Rather than using the generic openrouter/free endpoint, this implementation directly selects the NVIDIA Nemotron 3 Ultra 550B A55B

free model through OpenRouter. The exact model identifier used in the API request is nvidia/nemotron-3-ultra-550b-a55b:free.

Overview

This module demonstrates how to implement an AI chatbot within a Google Colab environment. Rather than binding directly to a single provider, this implementation leverages the official OpenAI Python SDK and redirects it to the **OpenRouter API**. OpenRouter acts as a unified proxy that normalizes the schema across models and providers. By routing requests through OpenRouter, developers gain access to over 500 models (including free-tier inference models) without having to rewrite their core integration logic.

Prerequisites & Environment Setup

To execute this script successfully, the environment must be correctly configured to handle secure credentials.

Required Accounts

- **Google Account:** To execute the notebook via Google Colab.
- **OpenRouter Account:** To generate an API key (available at openrouter.ai).

Secure Key Management

Hardcoding API keys is a security risk. This project utilizes Google Colab's native Secrets manager:

1. Navigate to the **Secrets** panel on the left sidebar of your Colab interface.
2. Click **Add new secret**.
3. Name the secret exactly: `OPENROUTER_API_KEY`.
4. Paste your OpenRouter API key into the **Value** field.
5. Toggle the **Notebook access** switch to allow the runtime to read the key.

Code:

```
from google.colab import userdata
from openai import OpenAI
```

```
OPENROUTER_API_KEY = userdata.get("OPENROUTER_API_KEY")
```

```
client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=OPENROUTER_API_KEY,
)
```

```
prompt_text = "Write a Recipe to create Chicken Tandoori"
```

```
completion_result = client.chat.completions.create(
    model="nvidia/nemotron-3-ultra-550b-a55b:free",
    messages=[
        {"role": "user", "content": prompt_text}
    ]
)

print(completion_result.choices[0].message.content)
```

Output :

```
[1] from google.colab import userdata
from openai import OpenAI

OPENROUTER_API_KEY = userdata.get("OPENROUTER_API_KEY")

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=OPENROUTER_API_KEY,
)

prompt_text = "Write a Recipe to create Chicken Tandoori"

completion_result = client.chat.completions.create(
    model="nvidia/nemotron-3-ultra-550b-a55b:free",
    messages=[
        {"role": "user", "content": prompt_text}
    ],
)
```

```
[1] print(completion_result.choices[0].message.content)

... # **Authentic Chicken Tandoori Recipe**
    *Yields: 4-6 servings | Prep: 20 mins + Marinating | Cook: 20-25 mins*

    ---

    ### **Ingredients**

    #### **The Chicken**
    * **2 lbs (1 kg) Chicken Leg Quarters** (or drumsticks/thighs, skinless, bone-in)
    * *Bone-in retains moisture best. Score the meat deeply (3-4 cuts per piece) to allow marinade penetration.*

    #### **Marinade 1: The Tenderizer (Apply First)**
    * **1 tbsp Lemon Juice** (fresh)
    * **1 tsp Salt** (or to taste)
    * **1 tsp Red Chili Powder** (Kashmiri for color, or mild paprika + pinch of cayenne for heat)
    * **% tsp Turmeric Powder**

    #### **Marinade 2: The Flavor Base (The "Masala")**
    * **1 cup (240g) Thick Greek Yogurt / Hung Curd** (strained 2+ hours-critical! Watery yogurt = steamed chicken)
    * **2 tbsp Mustard Oil** (traditional pungency; sub: vegetable oil + ½ tsp mustard powder)
    * **1 tsp Ginger-Garlic Paste** (freshly crushed)
    * *3 tsp Kashmiri Red Chili Powder (for vibrant red color without excessive heat)
```

Conclusion & Key Takeaways

Building this AI chatbot bridges the gap between theoretical Generative AI concepts and practical application. By implementing this code, we successfully applied several core topics from the syllabus in a real-world scenario:

- **Generative AI Fundamentals & LLMs:** We moved from theory to practice by directly interacting with a Large Language Model to generate text dynamically based on a user prompt.
- **Parameters, Training, and Inference:** Through the OpenRouter API, we executed an active inference request. The model processed our input prompt (context) and generated a response, demonstrating the end result of model training and text generation in real time.
- **NVIDIA Nemotron 3 Ultra via OpenRouter:** By explicitly using the nvidia/nemotron-3-ultra-550b-a55b:free model identifier, the application sends the prompt to the selected NVIDIA Nemotron model through OpenRouter and receives the generated response using the OpenAI-compatible chat completions interface.
- **Python for GenAI Applications:** We utilized Python as the primary application layer to securely manage credentials, structure the API payload using the standard openai library, and parse the complex JSON response.

Ultimately, this project serves as the foundational stepping stone. By successfully establishing an API connection and parsing an LLM's output, the groundwork is laid for advancing to more complex orchestration frameworks like **LangChain**, running **Local LLMs via Ollama**, and building full-scale AI agents.